Pipelines Basics

22 March 2026
13:36

Pipeline - art of PowerShell

The pipeline is used to filter the output.
Example:
By using the command : get-service we get huge output (say as 300 entries) to filter these we use pipeline. If we use the rule as display only "first 5 entries", we get our output as first 5 only rest all 295 entries are dropped
To count total services **: (get-service).count**

Using Pipeline

Here by using this command our output is filtered
Get-service | select-object -last 10

```
PS C:\Users\lenovo> get-service | select-object -last 10

Status   Name               DisplayName
------   ----               -----------
Running  WpnService         Windows Push Notifications System S...
Running  WpnUserService_... Windows Push Notifications User Ser...
Running  wscsvc             Security Center
Running  WSearch            Windows Search
Stopped  wuauserv           Windows Update
Stopped  WwanSvc            WWAN AutoConfig
Stopped  XblAuthManager     Xbox Live Auth Manager
Stopped  XblGameSave        Xbox Live Game Save
Stopped  XboxGipSvc         Xbox Accessory Management Service
Stopped  XboxNetApiSvc      Xbox Live Networking Service
```

To get only status and name columns mentioned in the output
Get-service | select-object -last 10 | select -object status, name

```
PS C:\Users\lenovo> get-service |select-object -last 10 | select-object status,name

 Status Name
 ------ ----
Running WpnService
Running WpnUserService_35e2470
Running wscsvc
Running WSearch
Stopped wuauserv
Stopped WwanSvc
Stopped XblAuthManager
Stopped XblGameSave
Stopped XboxGipSvc
Stopped XboxNetApiSvc
```

We can even store it in variable as shown

```
PS C:\Users\lenovo> $a = get-service
PS C:\Users\lenovo> $a |select-object -last 10 | select-object status,name

 Status Name
 ------ ----
Running WpnService
Running WpnUserService_35e2470
Running wscsvc
Running WSearch
Stopped wuauserv
Stopped WwanSvc
Stopped XblAuthManager
Stopped XblGameSave
Stopped XboxGipSvc
Stopped XboxNetApiSvc
```

Export all services in CSV Format

The command is: **$a | export-csv .\test.csv -notypeinformation**
Open it : **./test.csv**

A1 | Status

	A	B	C	D	E
1	Status	Name			
2	Running	WpnService			
3	Running	WpnUserService_35e2470			
4	Running	wscsvc			
5	Running	WSearch			
6	Stopped	wuauserv			
7	Stopped	WwanSvc			
8	Stopped	XblAuthManager			
9	Stopped	XblGameSave			
10	Stopped	XboxGipSvc			
11	Stopped	XboxNetApiSvc			
12					
13					
14					
15					
16					
17					

Services only in Running state using Where-object

Here we output of $a stored in $_ or $PSItem

$a | where-object{$_.status -eq "running"} | select-object -last 10

or

$a | where-object{$PSItem.status -eq "running"} | select-object -last 10

Here we used script block where we used to run the command inside the script block { }. Here we used -eq signifies equal sign where we need to use == but not -eq

```
PS C:\Users\lenovo> $a | where-object{$_.status -eq "running"} | select-object -last 5

 Status Name
 ------ ----
Running WpnService
Running WpnUserService_35e2470
Running wscsvc
Running WSearch
```

Format is using table format : **$a | where-object{$PSItem.status -eq "running"} | select-object -last 10 | format-table -Autosize**

Format using list format : **$a | where-object{$PSItem.status -eq "running"} | select-object -last 10 | format -list**

```
PS C:\Users\lenovo> $a = get-service
PS C:\Users\lenovo> $a |where-object{$_.status -eq "running"} |select-object -first 10

Status   Name               DisplayName
------   ----               -----------
Running  AdobeARMservice    Adobe Acrobat Update Service
Running  Appinfo            Application Information
Running  AppXSvc            AppX Deployment Service (AppXSVC)
Running  AudioEndpointBu... Windows Audio Endpoint Builder
Running  Audiosrv           Windows Audio
Running  BFE                Base Filtering Engine
Running  BrokerInfrastru... Background Tasks Infrastructure Ser...
Running  BTAGService        Bluetooth Audio Gateway Service
Running  BthAvctpSvc        AVCTP service
Running  bthserv            Bluetooth Support Service
```

Group-object

Here we used group-object for the status column of get-service where it groups status into 2 types stopped, running.
Here we see there are many services running in stopped and running rows along with count.
Choose such kind of property which is common like status

$a | group-object status

```
PS C:\Users\lenovo> $a | group-object status

Count Name                      Group
----- ----                      -----
  167 Stopped                   {AarSvc_35e2470, AJRouter, ALG, AppIDSvc...}
  110 Running                   {AdobeARMservice, Appinfo, AppXSvc, AudioEndpointBuilder...}
```

Measure - objects

Measure Objects are used to measure the properties.
$a | Measure object

```
PS C:\Users\lenovo> $a | measure-object


Count    : 277
Average  :
Sum      :
Maximum  :
Minimum  :
Property :
```

Compare-object

Here we compare 2 objects
We make $b as variable where we upload our demo file made in notepad. Then upload demo1 file in $c variable

$b = get-content .\demo.txt
$c =get-content.\demo1.txt
compare-object $b -differenceObject $c -includequal

```
PS C:\Users\lenovo> $b = get-content .\Documents\demo.txt
PS C:\Users\lenovo> $c = get-content .\Documents\demo1.txt
PS C:\Users\lenovo> compare-object $b -differenceObject $c -includeequal

InputObject  SideIndicator
-----------  -------------
hi           ==
hey          =>
yes          =>
you          =>
hello        <=
how are you? <=
good??       <=
```

== : the hi exist in both the files
=> = the hey exist in $b file(left side)
<= : the yes exist in $c file (right side)

$d = import-csv .\demo.csv
$d | select-object -first 10

```
PS C:\Users\lenovo> $d = import-csv .\Documents\demo.txt
PS C:\Users\lenovo> $d |select-object -First 10

hi
--
hello
how are you?
good??


PS C:\Users\lenovo> $d |select-object -First 10 | format-table -autosize

hi
--
hello
how are you?
good??
```